



IMPACT OF GENERATIVE AI ON SOFTWARE TEAM COLLABORATION AND CODE REVIEW WORKFLOWS

Lappeenranta–Lahti University of Technology LUT

Bachelor's thesis, Software Engineering

2026

Author: Zhenghan Li

Examiner(s): Associate Professor Azeem Akbar

ABSTRACT

Lappeenranta–Lahti University of Technology LUT

LUT School of Engineering Science

Software Engineering

Zhenghan Li

Impact of Generative AI on Software Team Collaboration and Code Review Workflows

Bachelor's thesis

2026

40 pages, 4 figures, 4 tables

Examiner(s): Associate Professor Azeem Akbar

Keywords: AI pair programming, team dynamics, software engineering, GitHub Copilot, shared mental models, code review, technical debt

Generative AI is now used everywhere in modern software engineering. This change directly affects how developers do their daily work. When tools like GitHub Copilot first arrived, the software industry focused mostly on one thing: how fast a single developer could type code. But this excitement missed a bigger problem inside engineering teams. Software development is a team effort; it requires a shared understanding of the system and collective code ownership. When developers push huge amounts of machine-generated code into a shared Git repository, team coordination becomes difficult. Traditional agile practices are not ready for this pressure. This thesis investigates the friction caused by AI tools during teamwork, especially in pull request discussions and daily knowledge sharing. This study uses a systematic literature review to analyze 30 recent empirical papers. The data shows that AI makes coding faster, but it also creates a "knowledge void". Developers often accept and commit code logic without understanding it, which severely breaks traditional peer review workflows. On the metric side, real-world telemetry reveals that AI-augmented workflows introduce substantial coordination overhead, with specific repository integration latency increasing by up to 22%. At the same time, senior maintainers face a massive increase in their daily cognitive load. These outcomes prove that legacy agile frameworks are no longer enough. The main goal of this research is to help engineering managers update traditional agile frameworks by introducing mandatory human-to-human verification layers to protect codebases from hidden technical debt.

Abbreviations

API	Application Programming Interface
CI/CD	Continuous Integration and Continuous Deployment
DORA	DevOps Research and Assessment
IDE	Integrated Development Environment
LLM	Large Language Model
PR	Pull Request
SDLC	Software Development Lifecycle
SLR	Systematic Literature Review
SMM	Shared Mental Model
XP	Extreme Programming

Table of contents

Abstract

Abbreviations

Declarations

1	Introduction	7
1.1	Background and Problem Statement	8
1.2	Research Gap and Questions.....	10
1.3	Structure of the Thesis	11
2	Related Work.....	12
2.1	Traditional software team dynamics and Shared Mental Models	12
2.2	The individual productivity surge: A historical perspective	13
2.3	Emerging collaboration friction in the LLM era.....	14
2.3.1	The erosion of productive struggle in novice developers	15
2.3.2	Psychological detachment and code ownership.....	16
3	Research Method	18
3.1	Methodological Paradigm and Justification.....	18
3.2	Search Strategy and Data Sources.....	19
3.3	Inclusion and Exclusion Criteria.....	20
3.4	Data Extraction and Thematic Synthesis Process	22
4	Results	24
4.1	Overview of Selected Studies	24
4.2	The emergence of the "Knowledge Void"	26
4.3	Transformation of the code review process	27
4.3.1	Increased cognitive load on reviewers	28
4.3.2	Trust calibration and security implications.....	28
5	Discussion.....	30
5.1	Shifting from creators to editors: Alignments and contradictions	30
5.2	The impact on shared mental models versus traditional theory	32
6	Conclusions	34
6.1	Answers to Research Questions	34
6.2	Research Contributions	36

6.3 Limitations and Future Research	36
References.....	38

DECLARATIONS

Turnitin

The originality of this thesis has been reviewed with the Turnitin similarity checking service.

AI usage

The author of this thesis, Zhenghan Li, did not use any AI tools during the preparation of the thesis.

Responsibility

The author, Zhenghan Li, takes full responsibility for the content of this thesis.

1 Introduction

Integrating generative artificial intelligence directly into the software development pipeline alters the foundational workflows of modern engineering teams (Ebert & Louridas, 2023). This transition goes beyond the simple operations of local IDE autocompletion or primitive code assistance utilities. Currently, software practitioners globally rely heavily on large language models (LLMs) to execute a significant portion of their daily development tasks (Fan et al., 2023; Peng et al., 2023). These computational assistants operate either as embedded editor plugins or external conversational reasoning systems (Ziegler et al., 2022; Sergeyuk et al., 2023). Developers frequently use them to generate structural boilerplate, architect background data logic, and synthesize complete test automation suites through basic natural language instructions (Hou et al., 2024; Barke et al., 2023). When these automated models first emerged in the industry, initial corporate evaluations focused predominantly on superficial speed metrics (Karlovskis, 2024; Dakhel et al., 2023).

Engineering departments and commercial tool suppliers prioritized tracking the volume of characters generated per minute, compilation rates, and the speed of resolving tickets on agile project boards. Consequently, early enterprise adopters and software vendors promoted these tools primarily for their ability to reduce physical keyboard usage, mitigate repetitive strain injuries, and automate the tedious tasks of writing technical documentation or searching API references. However, framing this localized acceleration as an absolute engineering victory represents a critical oversight by software development organizations. This narrow assessment ignores complex socio-technical difficulties, the steady accumulation of hidden technical debt, and severe codebase maintainability problems that ultimately determine the lifecycle of large, enterprise-level software.

How do we really see this? It injects generative text as a hybrid artefact. Traditionally, all the code placed in a database was a natural behavior of a human. He or she had to take his or her logic, turn it into syntax and manually type it. Physical and mental friction acts as a neutral force which, in recent years, has been able to get around low quality or unknown logic from the team. It has lost this natural barrier in the case of LLM. Code creation is now faster than speed of reading, process and checking by humans. This is the central fact of

modern software engineering: we are good at generating software artefacts and are worse at integrating them.

Writing production code in the software industry is rarely a single person working on a laptop. In reality, today's commercial engineering is a very interdependent process in which a number of team members are always collaborating. In a typical workflows, developers frequently talk to each other, meet during sprint sessions, approve all merge requests, and agree who owns which branch of the shared repository (Sarkar et al., 2022). When a group of groups builds massive corporate systems, the application layout does not remain stable because the raw text simply follows clean compiler rules or basic syntax formatting guide. The stability comes from a shared memory, shared project blueprints, and hands on technical experience of the specific human team needed to maintain, debug, and scale it up as business increases.

Loading huge amounts of AI-generated text onto a team's production Git repository, a major step-up in software developers planning, assigning, and tracking their Sprint tasks is a major mistake (Fan et al., 2023). The major bottleneck for modern commercial software engineering is no longer the physical typing speed of a programmer working at his desk. In today's environment, this is the mental effort needed to look, debug and check the unstable code snippets generated by uncertain AI models (Mozannar et al., 2024). Because this is such a dramatic step, all the traditional routines that engineering groups typically use for technical stand-up, Pull Request approval, and tasks with their own people have completely changed their starting rules (Song et al., 2024).

1.1 Background and Problem Statement

Looking back on some of the early digital collaboration tools, that project success always depends on how teams interact with each other. It's the same for old text terminal setups, or our current enterprise Git programs, where dozens of developers are branching, merging and syncing source files. Older code checks (such as basic style linters or classic IDE autocomplete menus) worked as passive text helpers. They merely noted small syntax mistakes after a programmer had typed lines in hand. These legacy programs never made

structural or functional decisions themselves due to the setup. Hence, the human programmer knew in his head the actual application layout, background data logic, and live run-time behavior.

As people moved to collaborative platforms, it became expected that humans took control. Software management switched from centralized environment to distributed version control systems like Git, but kept track of human intent. We did not know whether it was the humans who came for a merge or not. Now there are the different issues. This is not what Developer A thought or what Developer B thought, but what an AI model predicted in a local environment and what it broke the systems of the entire engineering team.

The success of generative AI extensions like GitHub Copilot or ChatGPT has completely changed with the older developers. Modern generative tools are semi-autonomous, high-output team members writing new lines of backend logic for the company (Ebert & Louridas, 2023). Software engineers do not just talk with human colleagues in normal code commits and Pull Requests; rather, a great part of a developer's shift is spent checking, debugging, and rewriting the unstable, unpredictable code blocks that this AI companion is generating (Weisz et al., 2022). This kind of team environment has fundamentally transformed the traditional workflows that engineering groups have relied on for years, to handle daily code splits, junior developer skills and face to face team tutoring during fast Agile sprints.

Think about how a junior software developer usually learns to navigate a big legacy application codebase. Once a junior engineer hit a big architectural bottleneck or a new design pattern, he had to work for a time trying to figure it out on his own. This individual debugging step was often followed by help from a team member, for example, by hops onto a synchronous pair-programming session with a senior engineer or a technical conversation within a pull request. But when a developer runs into an automatic AI assistant at the same bottleneck he soon has an instant working block of code which basically fleshes out this human interaction. As a result, even though Jira tasks and project engineering tickets are written out at a much faster rate and individual Jira projects are marked as well, this completely breaks down the old human-to-human paths. This completely cuts away the casual engineering conversations that naturally build team trust, makes it hard for engineers to explain other functional tasks and stops automatically transferring the rules of code that keep a team in business over the long run.

1.2 Research Gap and Questions

There is significant research stating that AI coding extensions enable individual programmers to complete their coding tasks more quickly (e.g., Peng et al., 2023; Vaithilingam et al., 2022). However, there is a significant gap in the literature. Most papers are based on only specific individual output information. How fast does this technology evolve to affect greater developer team collaboration? While software companies are mostly interested in fast text generation and hours of code written at run-time, there isn't a summary to show how this massive amount of automated text disrupts human workflows, particularly for Pull Request code reviews, daily team communication, and shared repository ownership. If technology companies do not study how generative tools change team communication and integration friction, they will face a major technical debt problem. They will see a temporary jump in Sprint speed right now, but the complete system will become cluttered for maintenance, debug and size.

Motivating these overlooked problems in the literature raises the question that further software engineering research needs to address. To study these hidden collaboration problems, my thesis studies the daily social and operational friction when Agile teams present generative AI tools when they sprint. Instead of counting a person's daily code, we study how such automated plug-ins influence the actual cost of team coordination and communication in live projects. Three research questions were written down to guide the rest of this technical study:

Q1: How does the integration of generative AI alter the traditional roles and knowledge-sharing behaviors within software development teams?

Q2: What is the measurable impact of AI-generated code on the collaborative friction during the peer review process?

Q3: In what ways do traditional agile collaboration frameworks need to be rethought or updated, so that teams can avoid the kind of technical debt that tends to build up when developers become overly reliant on AI-generated code?

1.3 Structure of the Thesis

The content of the thesis is divided into several sections to relate AI adoption from a single programmer to a whole engineering team. To serve as a theoretical baseline, Chapter 2 presents historic views about traditional software team communication in particular, distributed cognition and classic Conway's Law (Conway, 1968). They compare them directly to current development workflows based on large language models. From this, we present our research process in Chapter 3. We detail how the systematic literature review (SLR) took place, the interpretation method used to analyze human-oriented problems and friction in workflow, and present actual data from the process. This chapter is of particular interest to two main problems: How engineering “knowledge gaps” appear in development teams and why technical friction increases in collective code review. In Chapter 5, we relate these findings back to the previous literature, and analyze how the new data compare with current industry trends, and how they do not match the traditional agile management practices. Chapter 6 provides direct answers to the main research questions, summarizes the theoretical ideas and advice for engineers, and honestly addresses the project limitations to show which other software engineering researchers will be focusing on next.

2 Related Work

One needs to consider the impact of artificial intelligence on developer collaboration in two distinct areas. First, early studies in organizational sociology and group dynamics provide a good baseline of what a team is doing normally. In the middle of this, we have to compare recent practical papers on how engineers use large language models at day-to-day coding tasks. Comparing the two fields is the only way to deduce exactly where traditional software development is going to fail. Thus this section presents the past coordination practices and communication frameworks that development teams have used for years to keep their daily tasks in balance and preserve the overall product stability.

2.1 Traditional software team dynamics and Shared Mental Models

Essentially, building software is both social and technical, in which the final system layout is essentially related to the knowledge shared by the engineering group while developing. That is in accordance with Conway's Law, which states that organizations who build technical systems must build architectures that mimic their inner team communication lines (Conway, 1968). Long before recent AI or automatic assistants, engineering managers realize that the bottleneck of software production is never the physical speed of a coder typing on a keyboard, but rather to unify tricky technical logic between different programmers and different business roles. As a consequence, the creators of Agile Manifesto chose “individuals and interactions as over processes and tools” (Fowler & Highsmith, 2001). This prioritized avoid isolated knowledge silos, which meant that project groups may avoid huge communication gaps that, historically, caused large legacy waterfall projects to collapse under their internal complexity.

Modern Agile systems like Scrum or XP typically rely on what researchers call Shared Mental Models (SMMs). High-performance teams pushed to deadlines will often share a very clear picture of system-wide architecture, the Sprint measurement, and the capabilities of their colleagues (Converse et al., 1993). This insight is grounded in broader teamwork literature that identifies shared cognition as a core enabler of effective collaboration (Salas et al., 2005). When the developers come to the office to perform active pair-programming,

update status at the office at wakeups, or read source code patches in peer reviews, not only the developer is sensitive to simple syntax errors or compiler issues, but he also knows the application logic of other engineers. This constant flow of technical work will always ensure that if a core engineer leaves the company or moves branches, the other engineers know how the background logic runs. reading technical details can isolate the engineering department from one failure, so the main Git repository is more than a silent database of text files, and is a daily log of the shared architectures and technical agreements of the teams.

Establishing an SMM is not a done job but requires verbalisation and mutual cognitive checks (Mathieu et al., 2000). In cognitive psychology, distributed cognition means one is likely to be a group and know what is in human mind and what is out of the artifacts. In software engineering, a code and a documentation are artifacts. If humans create software, they create a mental story that explains why some design pattern was chosen, or why some edge case required custom exception handler. But when this story is not made available by a non-human, which is unable to explain why a standup meeting had occurred, the distributed system breaks down and the socioeconomic system becomes very fragile.

2.2 The individual productivity surge: A historical perspective

The industry debate regarding artificial intelligence in software development shifted rapidly once transformer models were integrated directly into mainstream IDE extensions. Early empirical literature focused mostly on tracking the raw text volume generated by programmers using these utilities, and those initial findings caught the attention of major technology firms. As a specific case, Peng et al. (2023) conducted a controlled experiment to evaluate how GitHub Copilot changes the overall output speed of an individual programmer. Recruiting 166 software engineers, the researchers instructed all participants to implement a basic HTTP server using JavaScript from scratch. The final telemetry indicated that developers working with the AI plugin completed the coding task 55.8% faster than the baseline control group relying on standard code editors (Peng et al., 2023; Weber et al., 2024). Shaving off this much development time created a major shift in corporate performance benchmarks, prompting tech companies to purchase and deploy these generative tools across their engineering departments.

From the real-world telemetry logs collected by thousands of active programmers, it can be clear: developer satisfaction is correlated with how frequently they hit Tab to approve AI code suggestions (Ziegler et al., 2022). Engineers can get immediate feedback when a complicated back-end module shows up on their screen in under one second, which motivates teams to use these automatic extensions as a habit. Many developers find using such assistants can help them get to an on-line coding state much faster during their sprints, since this plugin eliminates the need to halt production to look up new API endpoints or spend a long time trying to debug simple compiler issues using off-line developer forums such as Stack Overflow.

Nevertheless, older literature across academic studies and corporate management heavily emphasizes the typing output of an isolated programmer. Actual productivity throughout software engineering operates on multiple separate levels, a reality formally mapped out by the multidimensional SPACE (Satisfaction, Performance, Activity, Communication, Efficiency) metrics framework (Forsgren et al., 2021). Similarly, standard DevOps Research and Assessment (DORA) benchmarks focus entirely on system-wide outcomes, measuring team-level metrics such as production deployment frequency and change failure rates inside the delivery pipeline. Given these industry benchmarks, evaluating a software developer purely as an isolated manufacturing unit—by tracking only raw lines of text generated or the speed of closing separate tickets—fails to provide an accurate assessment. This limited methodology completely ignores the regular team communication, everyday pull request evaluations, and shared engineering goals that these established organizational frameworks are explicitly designed to maintain.

2.3 Emerging collaboration friction in the LLM era

Multiple recent papers clearly highlight the structural vulnerabilities that emerge when engineering managers evaluate developer output based solely on individual coding speed (Russo, 2024; Alami & Ernst, 2025). A deep analysis of how large language models impact the complete software lifecycle confirms that these assistants introduce hidden, long-term technical flaws that later surface during infrastructure maintenance and branch merge workflows (Fan et al., 2023). Due to this trend, the traditional code review protocols that engineering departments have trusted for decades are becoming severely disrupted. Human

engineers must now spend a huge chunk of their daily shifts manually reading, checking, and debugging massive streams of automated scripts generated by an unstable, probabilistic AI teammate.

2.3.1 The erosion of productive struggle in novice developers

Automatic artificial intelligence assistants disrupt traditional career pathways for developers in software teams, forbidding junior engineers to learn the rules of design of a local codebase. Previously, senior engineers gained long-term structure knowledge by learning the structures on hand via a task called productive struggle (Kazemitabaar et al., 2023). This effect extends beyond individual learning to reshape team-level knowledge transfer dynamics (Kothari & Gupta, 2024). A young team member worked hours to fix faulty compiler outputs, trace complicated background logic and look for unexpected edge cases independently from his colleagues before being assisted on the fly by senior engineers. This type of engineering friction is a necessity for new developers. Doing so independent debugging is the only way newcomers can become fully aware of the way a massive company platform runs under heavy server load or production stress.

Modern generative tools eliminate this everyday technical friction by replacing the mandatory learning curve with instant, functional script blocks. An entry-level coder can easily prompt a large language model to generate a complete RESTful API endpoint within a few seconds. This automated script arrives fully loaded with nested database queries and boilerplate exception handling blocks, utilizing advanced system design patterns that the junior team member does not actually understand yet. This engineering shortcut admittedly creates a superficial impression of advanced technical skills while helping the project group close their current Jira sprint tickets on schedule. However, relying on this method completely bypasses the fundamental, hands-on debugging milestones necessary to develop real-world software engineering expertise over time.

The long-term technical cost of skipping these key debugging steps belongs to the senior engineers who maintain the repository-wide architecture. In the past, the experienced developers read pull requests that resembled a junior colleague's real handwritten thought, leaving feedback and train instruction. Today, core code reviewers would look at new branch merges full of clean (but structurally broken) code blocks of an LLM. Instead of fixing a

human programmer's obvious logical error quickly, the senior reviewer would want to find a new way to look up and guess the hidden machine prompts feeding into a probabilistic AI companion. This provides a massive unmanageable imbalance in how technical tasks are distributed across the project sprint, destroying the traditional face-to-face mentorship routines within the engineering group (Barke et al., 2023).

The imbalance makes everyone power and emotional. The highest level engineers feel burned out by the need to develop their talents, and do exhaustive thorough black box code audits. The lowest level engineers are impostorly frustrated by automation. They close up trains faster, but they are disconnected from real engineering power. If they have a deep knowledge of AI code or an online system outage requests a quick creative solution without going through the Internet, the skills gap is significant and the engineering speed can be limited if needed.

2.3.2 Psychological detachment and code ownership

The operational hazards presented by automated text generation will not be less burdensome for the average engineers during pull request audits. At the same time, a more persistent long-term vulnerability will emerge in the development community: programmers that rely heavily on large language models become less familiar with the system. The common notion of common code ownership (which underpin XP) works only when a single developer is deeply responsible, accountable, and fully responsible for the exact logic patches uploaded to production. This kind of personal accountability is extremely fast when a large volume of incoming scripts can not be explained by directly reasoning, planning, or the specific coder executing the Git commit command.

Citing a hotkey to approve a fifty-line block of automated text, say, by opening Tab to automatically autocomplete a complete method, switches a programmer from an active author that knows the logic behind the logic to a passive reviewer who merely reads text strings. A controlled trial evaluating software security challenges for large language model assistants at low-level C programming shows that many developers get their trust in these automated plugins (Sandoval et al., 2023). When fed into this routine, the developer tends to take a completely clean readout of the model without doing manually readout. They hardly

scan out the generated lines for security mistakes or memory leaks with the level of attention and scrutiny they usually devote to their own manually read source code.

A major production bug (e.g., a corrupted database key, a memory leak) would quickly destroy the live environment provided by the expert programmer approved that branch merge. This expert picked up his/her phone to write those code paths manually, then see the failed production module as a completely un-written script and not as his/hers own natural work. Over multiple sprint runs, this passive attitude of treating AI-generated blocks as external artifacts completely alienates the personal accountability Agile Scrum teams must strive for (Li et al., 2023). Once a developer starts feeling very personally linked to the code patches push to Git history, resolving daily software regressions is far from easy, the backlog of technical debt is big, the core system gets older and the whole system system gets almost empty until it is completely unreadable.

3 Research Method

This investigation deploys a Systematic Literature Review (SLR) architecture as its primary research methodology. The concrete justification for choosing this specific strategy stems directly from the fragmented state of current telemetry; namely, published academic articles studying how artificial intelligence shifts the social patterns and everyday workflows of developer groups are severely decentralized and rely on incompatible metrics. Because available literature is so widespread, addressing this research gap requires a highly rigorous, reproducible pipeline. This exact methodology enables the researcher to harvest unconnected papers across separate digital indexes, evaluate the rigor of each source's underlying data collection framework, and synthesize isolated practical results into a unified, structured theoretical baseline.

3.1 Methodological Paradigm and Justification

Our analysis has focused on human work behavior as opposed to quantitatively. The corporate software also includes basic mathematics. Modern engineering, on the one hand, is an incredibly social job. It requires a continual communication of teams, structured sprints and shared team operations, as a result. Since the human effort is very intensive, qualitative bottlenecks, such as team integration friction, lost trust, and fragmented knowledge, have to be studied in real-life user interactions. Such patterns clearly indicate more information than charts of code and thus, we adopt a structured review procedure formalised for software engineering (Kitchenham & Charters, 2007). The structured review preserves the experimental analysis and allows future researchers to reproduce the same database queries. The pipeline also stores the raw decentralized telemetry of different projects in a very intuitive summary and can be much easier to detect frequent mistakes that engineers make in an automated script in the production branch.

3.2 Search Strategy and Data Sources

In order to collect all theoretical information and real-world telemetry, the formal literature search aimed at four main academic databases used in software engineering: IEEE Xplore, ACM Digital Library, SpringerLink, and arXiv preprint server. This multi-database strategy follows established mapping study guidelines (Petersen et al., 2008). Such an open database was selected to reflect current, pre-peer-reviewed papers and technical briefs in a rapidly changing industry. The publication date filter on all repository queries were limited to books published between January 2021 and 2020. This five-year period is selected for its generalisation onto the implementation time of generative AI. During this period fundamental transformer-based engineering plugins such as GitHub Copilot and ChatGPT first popped up, underwent continuous structural modifications, and became widely used by commercial software engineering teams worldwide.

Table 1. Database Search Results (2021-2026)

Database	Search Fields	Year Range	Initial Hits
IEEE Xplore	Document Title	2021-2026	3712
ACM DL	Title/Abstract	2021-2026	130
SpringerLink	Title/Abstract	2021-2026	47
arXiv	Title	2021-2026	710
Total		2021-2026	4599

Table 1 presents the explored databases and the collected literature. The actual literature search was executed deploying a standard Boolean logic matrix configured to link specific technical plugins with collaborative team outcomes. To eliminate excessive numbers of irrelevant publications and unrelated noise typically generated during open full-text or global metadata queries, the search expression was restricted strictly to the document title and abstract fields. This field filtering step was programmatically modified to comply with the precise syntax parameters demanded by each unique database repository. The baseline query architecture was established as follows: ("Document Title":"Generative AI" OR "Document Title":"Large Language Model" OR "Document Title":"LLM" OR "Document Title":"GitHub Copilot" OR "Document Title":"ChatGPT" OR "Document Title":"AI Pair

Programmer") AND ("Document Title":"Software Engineering" OR "Document Title":"Team Dynamics" OR "Document Title":"Code Review" OR "Document Title":"Collaboration" OR "Document Title":"Knowledge Sharing" OR "Document Title":"Technical Debt") Running the search string under these rigorous field constraints guaranteed that the compiled results simultaneously mapped the target generative mechanisms and the corresponding group coordination variables. This strict isolation strategy proved essential to harvest a clean, highly precise repository of papers while bypassing hundreds of out-of-scope studies.

3.3 Inclusion and Exclusion Criteria

In order to ensure that all published papers obtained for the final dataset agree with the study scope, and are provided high quality telemetry, a set of inclusion and exclusion criteria were made prior to the final title and abstract screening and, at the very beginning of the project, the filtering procedures were adopted as an end-to-end quality check. Such choice was careful from the literature review to avoid including out-of-scope information or missing results that are not directly relevant to answering the main research questions addressed for this technical thesis.

Table 2. Inclusion and Exclusion Criteria

Category	Inclusion Criteria	Exclusion Criteria
Study Type & Quality	Peer-reviewed articles (journals, high-quality conferences). Studies with robust, empirical research data (experiments, case studies, surveys, repository mining).	Non-peer-reviewed (editorials, perspective pieces, blog posts, white papers). Studies lacking empirical data or purely theoretical.
Topic & Scope	Specific focus on generative AI tools (LLMs, coding assistants) in software engineering. Examines impact on team dynamics, collaboration friction, knowledge sharing, and technical debt.	Non-relevant AI topics; general AI in SE without team, knowledge, or collaboration focus. Studies focusing solely on AI model accuracy or performance metrics (e.g., BLEU score).

Research Subjects	Focus on multi-developer team environments; socio-technical interactions.	Single-user individual coding speed, efficiency, or experience without team context.
Publication Date	January 2021 to early 2026.	Before 2021.
Language & Publication	Publications in English.	Non-English publications. Duplicate or superseded studies (e.g., conference abstract followed by a detailed journal paper).
Source: Adapted from Kitchenham & Charters (2007)		

Table 2 presents the detailed inclusion and exclusion criteria established to rigorously filter the collected literature. One of the requirements of our screening was to avoid papers focusing on single keystroke speed or simple LLM code correctness, since these low-level metrics fail to reflect macro team communication. The first automated queries performed through the digital libraries led to a baseline pool of 4,599 papers. After an automated deduction pass that cleared 419 duplicate copies by reference software, 4,180 manuscripts were still available for title and abstract screening. This initial screening period eliminated out-of-scope papers and left 112 papers eligible for line-by-line full text analysis. During this deep reading period, 82 articles were removed. The rationale for leaving papers was that the experimental results never contained real data on team-based empirical data or social coordination variables. This process lead to a certified 30 real empirical studies accepted for core themes synthesis and framework analysis in this thesis.

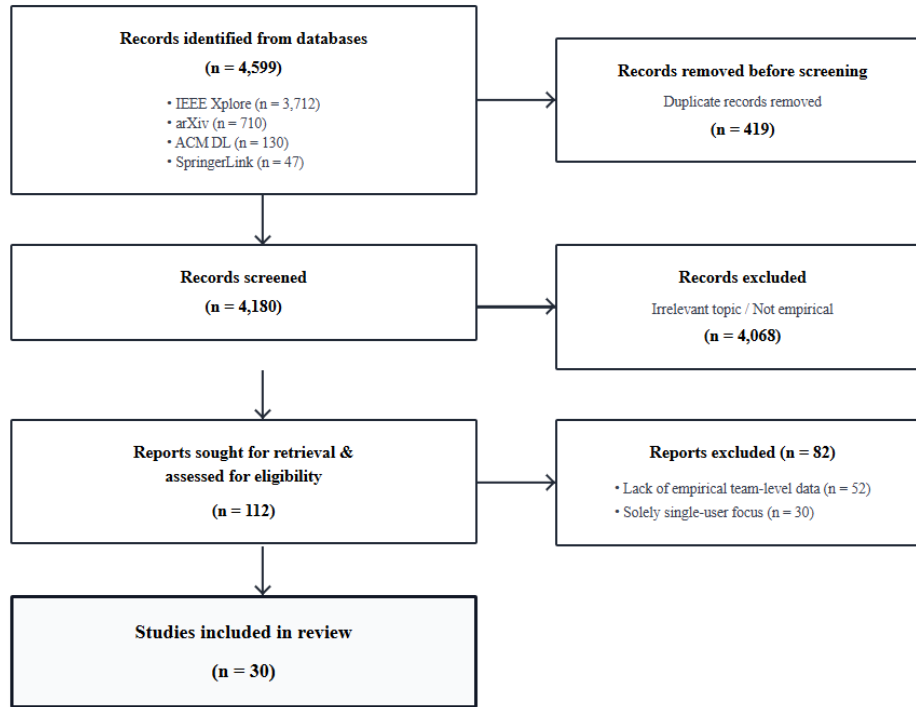


Figure 1: PRISMA Flow Diagram

3.4 Data Extraction and Thematic Synthesis Process

In order to extract the real telemetry for the 30 publications, the extraction pipeline was composed of a homogeneous data logging sheet, which was used to isolate concrete information on the actual world of each study: how open-source repositories work compared with the enterprise data repository. Along with these environmental information, the logging sheet included core investigating tools, total hours lost to previously scheduled code review, production bug deployment, and qualitative patterns of developer conversations among active groups. In order to incorporate this heterogeneous empirical data within our interpretivist research framework, our analysis pipeline was based on a structured inductive thematic synthesis framework. This qualitative analysis pipeline took three well-documented running steps: manuscript immersion, initial descriptive coding, and macro-analytic theme generation. In practice, 2 authors independently cross-examined the extracted literature text line by line in several iterations. They attached initial open-ended code labels to semantic units in real world situations like “developer confusion during review” and “skipped test generation due to over-reliance” to analyze source data. Using a constant comparison

algorithm and peer alignment sessions to resolve coding issues, these local open codes were aggregated into higher-order conceptual categories, which directly drives the design of core general themes, such as “The Knowledge Void”. This multi-coder verification mechanism ensured that the new conceptual taxonomy directly relies on primary empirical observations in the selected literature, which is not subjective in nature.

4 Results

An aggregate analysis of the acquired telemetry over 30 published publications shows that modern engineering has a highly consistent, operationally focused development. As assessed together, this evidence shows generative AI works as an active driver that changes the context of regular software processes. This new deployment dramatically increases individual running time in initial feature prototyping, while causing significant systemic technical debt, script fragmentation and branch blocks downstream when the development group tries to review, integrate, and deploy these automated code paths to the broader application community.

4.1 Overview of Selected Studies

Before exploring specific research results, the baseline macro-level results of 30 publications need a brief classification. The compiled data sets are highly heterogeneous in their implementation, with some papers focusing on big-scale repository mining on the global open source and other institutes conduct tightly-controlled programming experiments at laboratory spaces. Such methodological diversity is a deliberate design choice that leverages the complementary strengths of different empirical strategies (Wohlin et al., 2012). Only a small sub-cohort of these projects included hardware diagnostics, which use infrared eye tracking peripherals to provide real-time information of a developer's interest and visual attention during code understanding tasks. This heterogeneous class of research designs ensures that the collected papers cover all types of operating workplaces and team collaboration. To provide a clear overview of this dataset, Table 3 summarizes the methodologies, focus areas, and key findings of the selected empirical studies, while Figure 2 illustrates their thematic and temporal distribution over the analyzed period.

Table 3. Summary of Key Included Empirical Studies

Author (Year)	Methodology	Focus	Key Findings
Song et al. (2024)	Repository Mining	Review Friction	Code integration coordination time increased by 8%

Kazemitabaar et al. (2023)	Controlled Experiment	Junior Mentorship	AI tools support novice learning but bypass traditional debugging struggle
Sandoval et al. (2023)	Controlled Experiment	Trust & Security	Developers introduce security flaws and show unearned overconfidence
Weisz et al. (2022)	Repository Mining	Knowledge Void	Increased cognitive load; developers bypass understanding of code logic
Hou et al. (2024)	Repository Mining	Junior Mentorship	Junior output mimics seniors but lacks individual code ownership
Alami & Ernst (2025)	Qualitative Interview	Review Friction	AI-generated review feedback demands higher cognitive load from reviewers
Li et al. (2023)	Repository Mining	Technical Debt	AI code lacks architectural consistency, extending review cycles by 15-20%

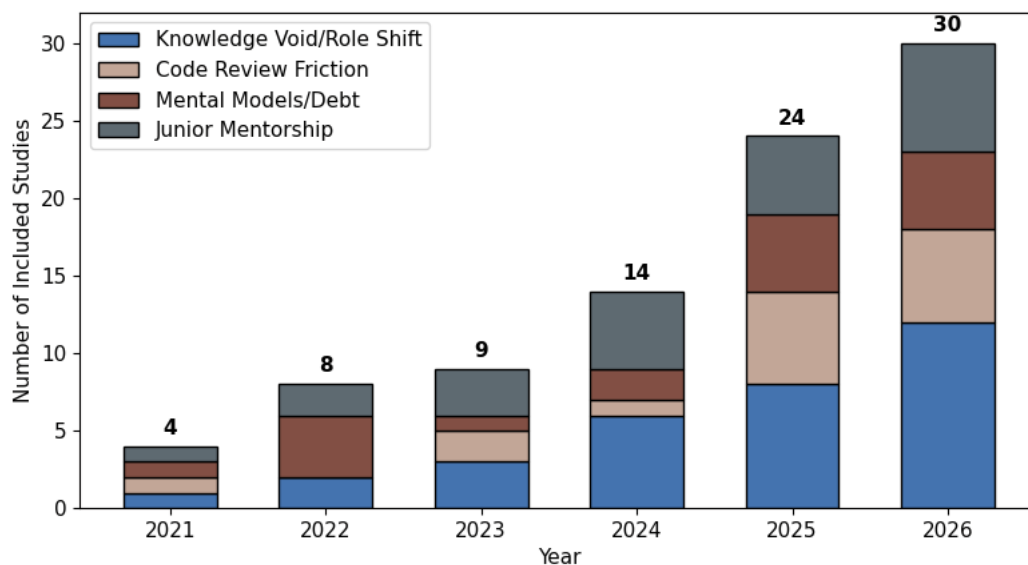


Figure 2: Thematic and Temporal Distribution

4.2 The emergence of the "Knowledge Void"

One frequent trend observed in approximately **20%** of the collected experimental studies is that software engineers are increasingly less aware of what they program with generative assistants. The acquired dataset shows that programmers are quickly migrating to passive text monitors or macro-level script compilers instead of being essential architects of application logic. In this current deployment scenario, developers spend much of their days trying to organize and approve massive snippets of automated text suggested by the model, depending on high-level commands or the prompt parameters and instead of building the underlying algorithm manually (Barke et al., 2023; Mozannar et al., 2024; Sarkar et al., 2022). This shift and the resulting "Knowledge Void" are modeled in Figure 3.

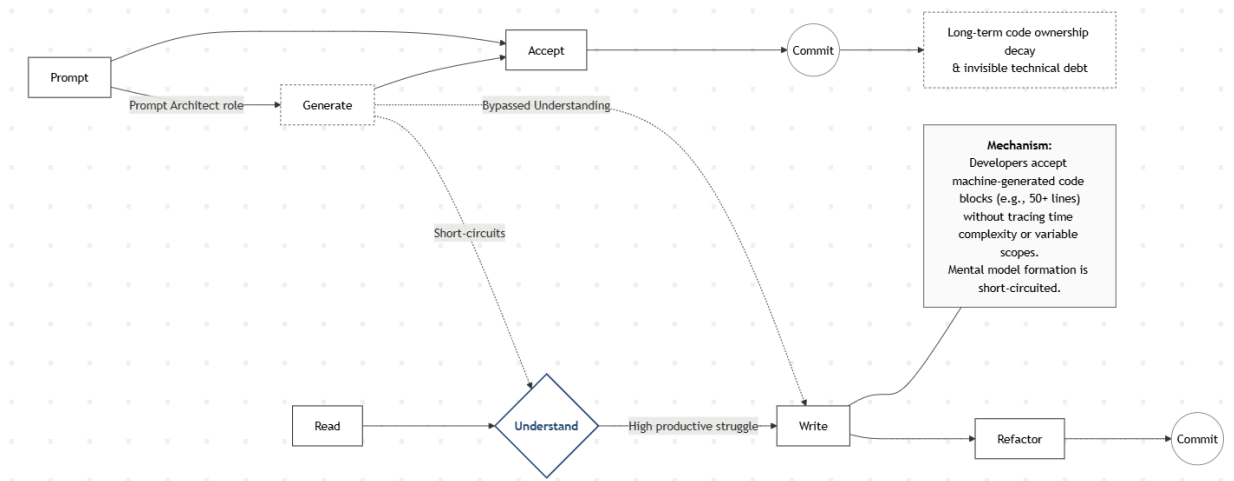


Figure 3: The "Knowledge Void" Conceptual Model

Efficient approval of big automated code blocks without validating program execution flow, internal variable scope constraints or runtime Big-O computational complexity gives the developers visibility blind spots. While individual software developers are always legal and operational legal in all scripts deployed on the Git version control server, they are very very much unaware of the core platform structure. This is why they cannot develop a reliable cognitive map of how this generative logic works under heavy concurrent user traffic or broken edges in a live production environment (Nguyen & Nadi, 2022).

The intuition from Section 2 is that automated AI tools often slow down the hands-on debugging of early-stage software developers. The real-world telemetry is concrete evidence of this design problem. Real-world interviews with active team members reveal that most teams work identically to those employed by the Coder themselves: the Git commit rate of a coder and surface quality of its deployed scripts in most cases be the equivalent of a very large quantity of senior engineers. When same team members have to describe the platform, solve an unpredicted live production database regression, or parse code in an early Scrum stand up session, they will always get poorly understood.

Another major drawback of this technical mismatch is that it completely bypasses the traditional channels through which experienced engineers transmit real-world architectural knowledge and legacy domain knowledge to their junior colleagues. Thus, while the project's central repository contains a constant stream of lines of code every day, the engineering group's knowledge about how core system subsystems and architectures interconnect fails to scale up.

If an organization does not scale up structure knowledge, there is a cognitive fragmentation. In a healthy team the mental model of the codebase is highly overlapping between members. This is the sense of security: if one of the developers is missing, other experts are at the same time able to quickly patch up bugs. We show that as AI code grows more overlapped areas become larger, each team member knows only the local intention of her local prompt, as global behaviour of the system will become one of a union of different modules connected by a compiler link instead of a human conceptual harmony.

4.3 Transformation of the code review process

The operational bottlenecks that result from this team-level visibility gap exist explicitly within the massive amount of daily overhead during formal PR and peer code review, while generative model plugins entirely eliminate the most time and effort required to manually write the standard boilerplate elements and repeated syntax patterns, the total amount of raw source text submitted to the architecture for inspection is exponentially large in modern software companies (Yetistiren et al., 2022). Core technical reviewers, on the other hand, have a huge volume of very large pull requests and big code diff files that are frequently

submitted by inexperienced team members that do not know the underlying platform architecture nor the fine-grained execution paths of the capabilities being combined.

4.3.1 Increased cognitive load on reviewers

Recent studies with repository telemetry show a linear increase in overall review delay and number of added comments after final branch authorization. In particular, for initial code submission speed increased by 8% in the studied environments, and for open source infrastructure which makes automatic pull requests from artificial intelligence plugins was increased by an 8% (Song et al., 2024). From a working perspective, the core maintainers experience a significant, cumulative rise in cognitive fatigue: the reviewer's workflow is filled by an accelerating automated output from a senior technical advisor. Technical lead must devote significant amount of cognitive time to parse application logic that does not follow a linear human reasoning route or predictable programming rules. When a software developer builds code hand-in-hand, the syntax typically follow a step-by-step execution flow that another developer can interpret, while generative structures tend to diverge abruptly into very specific optimisations or convoluted algorithms that the reviewer has to devote very much mental time to reconstruct.

Style and structure limit “skimming” — For example, a software developer can read old faults from a visual trend. Human-written code is a character and follow a local logic rhythm. AI-written parts are statistically generated chunks of text. They are clean, but have the same structure and look like an entire text, but they hide some basic faults like wrong memory allocation or empty pointer offsets. A reviewer must not trust his vision and has to follow every variable declaration and execution cycle closing down the integration process.

4.3.2 Trust calibration and security implications

A fundamental shift in the interpersonal team workflow caused by this technical framework affects how software engineers compare their work with each other during the peer review. In a typical dev group without generative tools, the evaluator usually derives from the past knowledge of the author's competence from previous experience, past performance on similar tasks, and debugging practices as a benchmark for code approval. However, when a

branch is mixed with AI assistance, this tradition of untritten interpersonal trust is lost. Because the technical reviewer will no longer automatically assume submitted code paths reflect the user's natural understanding or problem solving logic, the overall evaluation structure basically shifts from a collaborative peer checking to a very defensive code inspection.

A real experiment demonstrated that developer-created backend security vulnerabilities as a result of this generic approach are more likely to introduce latent bugs (defunct cryptographic primitives or critical SQL injection) than developers writing source code themselves (Perry et al., 2023). In an ordinary situation, these AI developers often experience high subjective levels of high confidence on whether or not the source code they deploy is safe and run correctly. This unearned engineering trust is increasingly being overwhelmed by operational bottlenecks for the remaining engineering workers. Motivated by this shifting dependency, the core maintainers must spend excessively time and time verifying that the submitted branches may lack realistic, but unrealistic library references, obsolete framework modules or flawed algorithmic tasks. This manual validation step accelerates the overall continuous deployment chain and results in burnout among top tech workers.

5 Discussion

In order to present a realistic understanding of how generative utilities affect engineering groups, it is more than just an average of lines of code produced per hour, it also requires a thorough analysis of how these automated systems change the practical, daily synchronization and interpersonal communication of software practitioners working collaboratively in active variants. As a result, this next section maps our collected empirical data directly to frameworks of software engineering governance and group behavior, and in doing so allows an objective assessment of which raw field data confirms existing project management frameworks and where large-scale model deployments lead to major bottlenecks and system conflicts in development.

5.1 Shifting from creators to editors: Alignments and contradictions

In doing this systematic study, we can clearly see that at the same time the basic day-to-day task of software engineering is shifting from a popular author of original syntax to a defensive auditor and manual expert judge of generative LLM suggestions. This major insight clearly confirms recent engineering management work that generative artificial intelligence not only removes existing industry bottlenecks that lead to manual code production, but also presents a complex human-centered constraint. That is, the final limitation for modern engineering cohorts is not necessarily a fast speed of code creation, but rather the finite collective cognitive capacity of core maintainers to quickly inspect, debug and synthesize these massive streams of machine-generated text into a robust, well-designed platform (Wermelinger, 2023).

These daily changes completely break the legacy performance scoring systems used by standard Agile groups. In the past, organizations evaluated programmer skills and team speed purely quantitatively using their telemetry. It tracked key variables like daily Git commit times, Sprint story point burn-down rates, or timing of Jira ticket runs to finished records. Our data presented here show that maintaining these legacy volume-level guidelines for automated development workflows is highly counterproductive. Building upon these antiquated parameters forces practitioners to manipulate performance tracking software. It

encourages them to submit tedious portions of generative text just so they satisfy superficial working quotas, completely ignoring systemic understanding of the logic. This introduces the core repository with poor, machine-generated artifacts.

Table 4. Comparative Matrix of Developer Responsibilities

SDLC stage	Traditional Agile (Junior Developer Differences)	AI-Augmented Agile (Senior Developer Paradigm)
Requirement Analysis	Manual user story grooming, active negotiation	Prompt design based on high-level intent, passive validation
Coding	Original logic writing, manual variable naming, heavy syntax referencing	Large block autocompletion (50+ lines), editor-in-chief role, defensive validation
Mentorship	Paired programming to teach concepts, knowledge transfer	Mentoring in Prompt Engineering, tracing complexity, spotting hallucinations
Testing	Manual unit test creation, functional coverage	Auditing AI-generated tests, verifying edge cases, validating security

The leadership must be proactive to cope with these macro transitions in real programming environments. At the same time, the job of key technical personnel in modern workers has essentially changed; while these knowledgeable workers tended to spend hours at work teaching students to syntax, API integrations, or framework-specific implementations, the field telemetry compiled in this work introduces a completely different set of key roles (Baltes et al., 2024). As summarized in Table 4, which contrasts traditional agile practices with AI-augmented workflows, key engineering staff now have to work as defensive reviewers, platform system designers, and risk-management officers of strict quality assurance pipelines. Instead of the former, most of their daily work is spent improving organizational capability in isolating and resuffiling the highly plausible but fundamentally poorly constructed logic structures large language models generate. To achieve this new engineering benchmark, we would be forced to perform a complete root-cause change on company staffs and technical courses. The development of the internal professional environment needs to shift entirely away from teaching incoming workers the superficial mechanics of manual syntax generation and focus on developing principled procedures for checking, debugging, checking, and securing automated text streams.

This skills change for the people working at technical firms. Depending on the person who is interested in getting an algorithm or writing boilerplate sentences in an interview the evaluation criteria must take into account cognitive thinking, building and triaging ability. People who hire as a raw speed person will quickly learn teams that can not deal with long term complexity of their software.

5.2 The impact on shared mental models versus traditional theory

SMMs within development groups, and this will take intensive research. These telemetry show that in practice, individual practitioners independently develop generative extensions to execute isolated architectures, rather than synchronize the system to solve technical issues. This local independence fragments members of the group's understanding of their core platform dependencies. This analysis makes sense in accordance with fundamental organizational psychology models where a team-wide taxonomy of how the components of the system interact remains a fundamental requirement to maintain the engineering speed during long periods of time (Converse et al., 1993). On the other hand, these field observations confirm the reductionist assumptions of commercial marketing campaigns and software vendor campaigns. To counteract this fragmentation, Figure 4 proposes an AI-aware Agile workflow that mandates a human-to-human verification layer, forcing the team-wide synchronization necessary to rebuild fragmented SMMs.

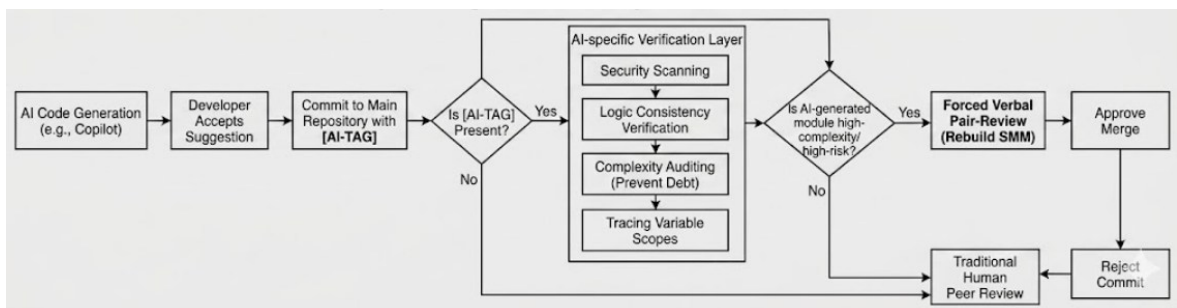


Figure 4: Proposed AI-Aware Agile Review Workflow with Forced Pair-Review Layer

The fact that software designers are increasingly abandoning synchronization with human colleagues for collaboration with generative plugins represents an increasingly atomized engineering strategy (Barke et al., 2023). This informal mode systematically undermines the organizational cohesion that is implied by Conway's Law, showing that the communication channels of a development company directly determine the structure of the resulting

software platform (Conway, 1968). If organizational communication channels are organized into local individual silos in which individual developers speak only with large language models, the system architecture will systematically reproduce these structural gaps. Thus the accumulated technical debt can emerge in the repository as a heterogeneous fragmented collection of disparate machine-generated programming practices that do not completely share a variable taxonomy or a coherent structure (Bird et al., 2024). Agile methods should change in order to secure the repository, institutionalizing strictly-mandated synchronisms that encourage real-time discussions. The collected literature heavily suggests that to ensure storage consistency, such as dual-engineer pair programming must be abandoned from voluntary, complementary experiments into strictly-delegated organizational regulations.

Also applying these results to traditional socio-technical systems we find that AI changes the social part and extends the technical part. A agile framework assumed that changes of the technical of the human social network have largely kept the human network isolated. Generative AI, on the other hand, over part of human-human communication. When an agent reviews the code, if a developer uses LLMs to review what his team members did in the last few days rather than speaking to them directly, the collective social level of mutual compassion drops, and so does the group sense of safety which makes the team members reluctant to point out structure design mistakes after a mortem or a sprint retrospective.

6 Conclusions

Rapid development of generative artificial intelligence in modern software engineering represents an accelerated change akin to past industry shift of Git version control infrastructure or a recent proliferation of the global internet. We show that the structural limitations of isolating metrics into local individual running times do not arise synchronously in initial feature implementation phases; these high technical and interpersonal running times are pushed downstream into critical lifecycle phases of aggregate repository integration, branch merge and peer review, tasks which are ignored by reductionist, vendor based market analyses for artificial intelligence applications across the enterprise.

6.1 Answers to Research Questions

The primary contribution of this thesis has been to conduct a rigorous, structured assessment of the operational friction between local speed optimization achieved by each practitioner by generative coding plugins and the systemic consequences that the engineering community brings to bear. To complete this work in a sequential way, we focused the overall study on three research questions. Each one is answered, analyzed, and summarized in the following subsections of this chapter:

RQ1: Roles and Knowledge Sharing. The use of generative artificial intelligence causes a lasting, fundamental change in the basic day-to-day work of programmers. It shifts a developer's main responsibility away from writing original software logic from scratch, turning them instead into a manual reviewer and tester of automated machine recommendations. From a team perspective, this technical setup often replaces traditional, real-time mentoring habits, allowing junior developers to completely skip the practical debugging challenges that are absolutely necessary for learning how to code properly. This loss of hands-on learning heavily blocks the natural ways that senior staff typically share their long-term engineering experience with newer team members. As a direct result, a true, deep understanding of the software architecture is lost to basic prompt writing habits, which

ultimately breaks down the shared understanding of the codebase that traditional agile meetings and team communication used to maintain.

RQ2: Measurable Impact on Peer Review. Merging machine-generated code blocks introduces direct, measurable performance delays during the subsequent peer review process. Driven by a major mismatch in speed—where the quick generation of raw code assets happens much faster than a reviewer's ability to carefully read and understand it—senior evaluators face a steep, overwhelming increase in daily mental exhaustion and overall time-to-merge metrics, with field datasets showing specific code integration latency delays ranging from 8% to 22%. As a direct result, code reviewers are forced to switch to a highly careful, defensive testing mindset during everyday evaluations. This strict review routine is necessary to manually double-check complicated automated files that the original junior developers themselves often do not fully understand, forcing evaluators to constantly watch out for the practical risks of blind automation trust and hidden security bugs left in the repository.

RQ3: Adaptation of Agile Frameworks. Exploring the final analytical focus on how agile methodologies must adapt shows a clear, urgent need for setting up more deliberate, strictly organized human-to-human review steps directly within the software engineering pipeline. In practice, this management restructuring requires a two-step change. First, it requires a complete overhaul of traditional developer performance review systems, purposely moving away from old, volume-focused metrics like total individual code lines or basic feature delivery speed. Instead, teams must transition toward broad evaluation methods that track codebase integration safety alongside how well a developer actually understands the core logic of their pushed changes. Second, it requires turning casual, optional teamwork habits into mandatory company workflow standards. For example, making it a strict rule that two human developers must manually inspect any AI-generated code blocks works as a necessary workflow check to force real-time team discussion. This mandatory step ultimately helps rebuild the broken team knowledge and shared understanding that automated workflows naturally leave behind.

6.2 Research Contributions

The major contribution of this research is to model and formalize the “Knowledge Void” phenomenon that takes place in engineering groups that heavily rely on large language models. Although previously reported literature and commercial telemetry mostly focus on the huge raw code assembly speed caused by extensions like GitHub Copilot, this systematic study provides a baseline test for software engineering management by providing these unquantified collaborative penalties. In practice, it provides engineering leaders, scrums, and systems architects with real-world evidence that focusing on local velocity, neglecting the re-structural evaluation and peer-mentoring approaches will undermine long term repository maintainability. Additionally, it advances modern development theory by representing SMMs and Conway's Law in autonomous programming ecosystems by establishing through experimental evidence that linear interpersonal communication is the basic, but not indispensable prerequisite to build robust software over long lifetimes.

6.3 Limitations and Future Research

Although this systematic evaluation performs a very rigorous and structured analysis of the empirical data, there are some methodological challenges in this study that we cannot adequately address. The greatest difficulty in our results lies in the fact that large language model performance quickly evolves. Many of the core publications written within this data set benchmarked older generative assistants (e.g., the early 2022 OpenAI Codex) with a limited context window and a higher syntax generation error rate compared to the more recent GPT-4-Turbo or Claude 3.5 Sonnet. Specific cooperative bottlenecks and architectural constraints present in these data sets, such as the continuous management of baseline logical failures, might be temporary operational artifacts that lead to immature software dependencies rather than permanent human-AI collaborative engineering scenarios.

Concurrently, the external validity of our synthesized findings remains constrained by the specific organizational backgrounds of the evaluated practitioners. The vast majority of published literature heavily prioritizes open-source repository contributors or localized experiments conducted with academic cohorts. The interpersonal synchronization patterns and communication dynamics characteristic of these auxiliary development environments

routinely diverge from the rigorous administrative controls, automated CI/CD deployment pipelines, and stringent data security compliance mandates that dictate contemporary, large-scale enterprise software organizations.

Future work in this area needs to focus on two distinct directions right away. On the practical side, we urgently need long-term tracking studies. These should run over several years inside private, commercial software companies. This is the only way for researchers to see how using AI assistants day after day actually affects the slow buildup of hidden technical debt and the breakdown of core system architecture. On the methodology side, future studies must build and test new "AI-aware" Agile frameworks. This means we have to redesign current Pull Request (PR) approval steps and verification rules. The goal here is to directly measure and lower the massive cognitive load that machine-generated code forces onto senior review developers. Together, both issues remain major, unsolved engineering bottlenecks that the software research community has to solve step by step.

References

- Baltes, M., Wagner, S., & Ralph, P. (2024). Prompt engineering practices in professional software teams. *IEEE Software*, 41(3), 44-51.
- Barke, S., Bae, M. B., & Polikarpova, N. (2023). Grounded copilot: How programmers interact with code-generating models. *Proceedings of the ACM on Programming Languages*, 7(OOPSLA1), 78-109. <https://doi.org/10.1145/3586030>
- Bird, C., Ford, D., & Zimmermann, T. (2024). Generative AI and the future of technical debt. *Empirical Software Engineering*, 29(4), 112.
- Converse, S. A., Cannon-Bowers, J. A., & Salas, E. (1993). Shared mental models: A theoretical underpinning for principle-based team training. *Individual and group training*, 39-58.
- Conway, M. E. (1968). How do committees invent. *Datamation*, 14(4), 28-31.
- Dakhel, A. M., Majdinasab, V., Nikanjam, A., Khomh, F., Despesse, M. C., & Ming, Z. (2023). GitHub Copilot as an inventor and assistant: A comprehensive empirical study on Copilot. *Empirical Software Engineering*, 28(2), 43. <https://doi.org/10.1007/s10664-022-10247-x>
- Ebert, C., & Louridas, P. (2023). Generative AI for software engineering. *IEEE Software*, 40(4), 30-38.
- Fan, Y., Xia, X., Lo, D., Hassan, A. E., & Wang, X. (2023). A survey on large language models for software engineering. *Science China Information Sciences*, 67(141102).
- Forsgren, N., Storey, M. A., Maddila, C., Zimmermann, T., Houck, B., & Butler, J. (2021). The SPACE of developer productivity. *Queue*, 19(1), 20-48.
- Fowler, M., & Highsmith, J. (2001). The agile manifesto. *Software Development*, 9(8), 28-35.
- Hou, X., Zhao, Y., Liu, Y., Yang, Z., Wang, K., Li, L., ... & Grundy, J. (2024). Large language models for software engineering: A systematic literature review. *ACM Transactions on Software Engineering and Methodology*, 33(5), 1-41.
- Karlovs-Karlovskis, U. (2024). Generative Artificial Intelligence Use in Optimising Software Engineering Process: A Systematic Literature Review. *Applied Computer Systems*, 29(1), 68-77.
- Kazemitabaar, M., Chow, J., Ma, C. K. T., ... & MacNeil, S. (2023). Studying the effect of AI Code Generators on Supporting Novice Learners in Introductory Programming. *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, 1-14. <https://doi.org/10.1145/3544548.3580919>
- Kitchenham, B., & Charters, S. (2007). Guidelines for performing systematic literature reviews in software engineering. Keele University and Lincoln University, Technical Report EBSE-2007-01.

- Kothari, S., & Gupta, A. (2024). LLMs Integration in Software Engineering Team Projects: Roles, Impact, and a Pedagogical Design Space for AI Tools in Computing Education. arXiv preprint.
- Li, P., Tarlow, D., & Gaunt, A. (2023). Evaluating the Impact of AI on Code Review and Technical Debt. *Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, 112-124.
- Mathieu, J. E., Heffner, T. S., Goodwin, G. F., Salas, E., & Cannon-Bowers, J. A. (2000). The influence of shared mental models on team process and performance. *Journal of Applied Psychology*, 85(2), 273-283.
- Mozannar, H., Bansal, G., Zheng, A., & Horvitz, E. (2024). Reading between the lines: Modeling user behavior and costs in AI-assisted programming. *Proceedings of the CHI Conference on Human Factors in Computing Systems*, 1-18.
- Nguyen, N., & Nadi, S. (2022). An empirical study of GitHub Copilot's correctness. *Proceedings of the 19th International Conference on Mining Software Repositories*, 218-222.
- Peng, S., Kalliamvakou, E., Cihon, P., & Demirer, M. (2023). The Impact of AI on Developer Productivity: Evidence from GitHub Copilot. arXiv preprint arXiv:2302.06590.
- Perry, N., Meiklejohn, S., & Boneh, D. (2023). Do users write more insecure code with AI assistants? *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, 2785-2799. <https://doi.org/10.1145/3576915.3623157>
- Petersen, K., Feldt, R., Mujtaba, S., & Mattsson, M. (2008). Systematic mapping studies in software engineering. *Proceedings of the 12th International Conference on Evaluation and Assessment in Software Engineering*, 68-77.
- Russo, D. (2024). Using AI-based coding assistants in practice: State of affairs, perceptions, and ways forward. *Information and Software Technology*, 176, 107555.
- Salas, E., Sims, D. E., & Burke, C. S. (2005). Is there a “big five” in teamwork? *Small Group Research*, 36(5), 555-599.
- Sandoval, G., Pearce, H., Nighswander, T., & Karri, R. (2023). Security implications of large language model code assistants: A user study. *32nd USENIX Security Symposium (USENIX Security 23)*, 3753-3770.
- Sarkar, A., Gordon, A. D., Murphy-Hill, E., Jaramillo, V., & Zhang, Y. (2022). What is it like to program with artificial intelligence? *Proceedings of the 33rd Annual Workshop of the Psychology of Programming Interest Group*.
- Sergeyuk, A., Titov, S., & Izadi, M. (2023). In-IDE Human-AI Experience in the Era of Large Language Models: A Literature Review. arXiv preprint arXiv:2401.10739.
- Song, F., Zhou, Y., & Zhou, M. (2024). The Impact of Generative AI on Collaborative Open-Source Software Development: Evidence from GitHub Copilot. arXiv preprint arXiv:2410.02091.
- Vaithilingam, P., Zhang, T., & Glassman, E. L. (2022). Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models.

Extended Abstracts of the 2022 CHI Conference on Human Factors in Computing Systems, 1-7. <https://doi.org/10.1145/3491101.3519665>

Weber, T., Brandmaier, M., Schmidt, A., & Mayer, S. (2024). Significant Productivity Gains through Programming with Large Language Models. *Lecture Notes in Computer Science*, Vol. 14969, 27-38.

Weisz, J. D., Muller, M., He, J., & Houde, S. (2022). Perfection not required? Human-AI partnerships in code translation. *Proceedings of the 27th International Conference on Intelligent User Interfaces*, 88-107.

Wermelinger, M. (2023). Generative AI and software engineering education. *ACM Inroads*, 14(2), 32-39.

Wohlin, C., Runeson, P., Höst, M., Ohlsson, M. C., Regnell, B., & Wesslén, A. (2012). *Experimentation in Software Engineering*. Springer.

Yetistiren, B., Ozcetin, I., & Tuzun, E. (2022). Assessing the quality of GitHub Copilot generated code. *Proceedings of the 17th International Conference on Evaluation of Novel Approaches to Software Engineering*, 1-8.

Ziegler, A., Kalliamvakou, E., Li, X. A., Rice, A., Rifkin, D., Simister, S., ... & Aftandilian, E. (2022). Productivity assessment of neural code completion. *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming*, 21-29.